

CS2109S

AY25/26 Sem 1

by ngmh

Introduction

- PEAS: Performance Measure, Environment, Actuators, Sensors
- A rational agent chooses actions that maximise performance measure
- Agent perceps with sensors and perform actions with actuators
- Environments:
 - Fully Observable (v/s Partially Observable): Agent's sensors give it access to complete state of environment, no need to keep track of state and no hidden information
 - Single agent (v/s Multi agent): Presence of multiple agents in environment
 - Deterministic (v/s Stochastic): Next state of environment is completely determined by current state and action executed by agent; if other rational agents actions matter it is also strategic, ignore uncertainty from actions of other agents even though agent cannot predict others actions
 - Deterministic: Given a state and set of actions, we can precisely determine the next state
 - Stochastic: Given a state and set of actions, we cannot determine the exact next state, but we know the probability of reaching each possible next state
 - Episodic (v/s Sequential): Experience is divided into atomic episodes of agent perceiving then performing an action, and decision making is only dependent on the current episode; as opposed to current decision affecting future decisions
 - Static (v/s Dynamic): Environment is unchanged while an agent is deliberating; semi-dynamic if environment does not change but agent's performance score does
 - Discrete (v/s Continuous): Limited number of distinct, clearly defined percepts and actions
 - Tic-Tac-Toe: Fully Observable, Deterministic, Strategic, Static, Sequential, Multi-agent

Intelligent Agents

- Agent program implements agent function that maps from percept histories to actions
- Exploitation v/s Exploration: Balance between learning more about world and maximising gain based on current knowledge

Designing an Agent

- Search Problem: Goal is to find a state or path by exploring various possibilities
- Agent needs a goal, model of environment, and search algorithm
- Problem Formulation: States, Initial State, Goal / Test State, Actions, Transition Model, Action Cost Function
- Representation Invariant: Ensure that abstract states have corresponding concrete states with restrictions
- Every state has a set of actions, with branching factor b if number of actions is less than or equal to b
- Transition Model: Uses transition function to produce next

state given state and action

Searching

- Path Cost: Cost of a path from any state to any state
- Optimal Path Cost: Cost of lowest cost path from any state to any state
- Complexity: Time (Number of nodes generated or expanded) or Space (Maximum number of nodes in memory)
- Completeness: Algorithm will find a solution for every problem instance (if one exists)
- Optimal: When an algorithm outputs a solution, it is guaranteed to be optimal (the best possible)

Uninformed Search

BFS

- Maintain a frontier queue, and repeatedly expand shallowest unexpanded node
- Time and Space Complexity: Exponential wrt depth of optimal solution
- Complete if graph is finite
- Optimal if step cost is same everywhere

Uniform Cost Search

- BFS but with non-identical edge weights
- Use a priority queue with path cost instead
- Time and Space Complexity: Exponential wrt tier of optimal solution
- Complete if > 0 step cost everywhere and finite total cost
- Optimal if > 0 step cost everywhere

DFS

- Use a stack to maintain nodes, expand deeper nodes first
- Time Complexity: Exponential wrt max depth of tree
- Space Complexity: Polynomial wrt max depth of tree
- Not Complete when depth of search tree is infinite
- Not Optimal when solution is at shallower depth
- Complete with visited memory

Informed Search

- Has information on search e.g. how good a state is
- Heuristic: Estimate of optimal path cost from any state to the goal state, e.g. straight line distance or Manhattan distance
- Best First Search: Use a priority queue, and evaluate in order of evaluation function of a state
- A* Search: Evaluation function $f(n) = g(n) + h(n)$, where g is the cost to reach node n , and h is the estimated cost from n to goal
- Time and Space Complexity: Exponential
- Complete if edge costs are positive and branching factor is finite
- Optimality depends on heuristic
- Admissible Heuristic: For every node, $h(n) \leq h^*(n)$, where $h^*(n)$ is the optimal path cost to the goal state from n ; heuristic never overestimates cost to reach goal
- Theorem: If $h(n)$ is admissible, A* without visited memory is optimal
- Consistent Heuristic: For every node, every successor n' generated by action a satisfies $h(n) \leq c(n, a, n') + h(n')$, $h(G) = 0$, essentially triangle inequality

- Consistency implies Admissibility
- Theorem: If $h(n)$ is consistent, A* with visited memory is optimal
- Relaxed Problem: Has fewer restrictions, optimal solution to relaxed problem is an admissible heuristic
- If $h_1(n) \geq h_2(n)$ for all nodes, then h_1 dominates h_2 , and is better for search if admissible

Search Strategies

- Some search algorithms might not terminate without visited memory
- Can set a strict limit on resources an algorithm can use
- Gives rise to a search with polynomial memory which is complete and optimal
- Depth Limited Search: Limit search depth, backtrack when limit is hit
- Time Complexity: Exponential wrt depth limit
- Space Complexity: Polynomial wrt depth limit if used with DFS
- Not complete, not optimal (if used with DFS)
- Iterative Deepening Search: Search with increasing depth limits, returning a solution if found
- Time Complexity: Exponential wrt depth of optimal solution
- Space Complexity: Polynomial if used with DFS
- Complete, optimal (if step cost is same everywhere)

Local Search

- Traverse state space by considering only locally-reachable states, repeatedly moving to better ones
- Typically incomplete and suboptimal
- Any-time property: Longer runtimes often have better solutions
- States represent potential solutions which might not be actual states
- Goal Test checks if a current state is the desired solution
- Successor Function generates neighboring states by modifying the current state
- Evaluation Function: Mathematical function used to assess the quality or desirability of a state which we either want to minimise or maximise
- Hill Climbing: Repeatedly move to the best successor based on eval until the current state is the best
- Shoulder: Objective function is flat and hard to move past

Adversarial Search

- Multi-agent problems involve multiple agents which could be either cooperative or competitive
- Adversarial Games: Agents compete against each other; Zero-Sum: One player's gain is the other player's loss
- Turn-Taking Games: Next state of environment also depends on action of strategic opponents
- Game Tree represents all possible states and moves, and outcome is displayed below each terminal node
- Utility Function: Outputs value of a state from the perspective of our agent, which wants to maximise it
- Utility of non-terminal states is typically not known in advance and must be computed

Minimax

- Assumes all players play optimally
- Player A: max, Player B: min
- Time Complexity: Exponential wrt maximum depth

- Space Complexity: Polynomial
- Complete if tree is finite
- Optimal if against optimal opponent
- Theorem: Minimax computes the optimal strategy for both players assuming both play optimally
- Theorem: If the opponent deviates from optimal play, the utility obtained by Player A is never less than the utility they would obtain against an optimal opponent
- Minimax explores the entire game tree in a depth first manner which can be costly

Alpha Beta Pruning

- A variant of minimax which decreases the number of nodes that need to be evaluated
- α : Highest value for A so far
- β : Lowest value for A so far
- On Player B's turn, if an action leads to a worse outcome, then the rest of the actions are irrelevant since the node will have at most that outcome which Player A does not want
- On Player A's turn, if an action leads to a better outcome, then the rest of the actions are irrelevant since the node will have at least that outcome which Player B is not interested in
- Initialise $\alpha = -\infty, \beta = +\infty$
- At a max node, set v to be $\max(v, v_i)$. Then set $\alpha = \max(\alpha, v)$. If $v \geq \beta$, prune remaining nodes.
- At a min node, set v to be $\min(v, v_i)$. Then set $\beta = \min(\beta, v)$. If $v \leq \alpha$, prune remaining nodes.
- α, β are not returned by reference!
- Theorem: Alpha-Beta pruning does not alter the minimax value and optimal move computed; it only reduces number of nodes evaluated
- Theorem: With good move ordering, runs in $O(b^{\frac{m}{2}})$
- Cutoff: Apply a strategy to halt search in middle due to computational constraints
- Estimate value of mid-game states with evaluation function which assigns a score to a game state
- A heuristic function estimates utility of a state, which should fall somewhere between the minimum and maximum possible utility for any state
- Derived from relaxed game, expert knowledge, or learnt from data
- Theorem: In a finite two-player, zero-sum game, minimax with cutoff and an evaluation function returns a move that is optimal with respect to the evaluation function at the cutoff (but not necessarily the true value)

```
def max_value(state, a, b):
    v = -INF // +INF
    for next_state in expand(state):
        v = max(v, min_value(next_state, a, b))
        // min(max)
        a = max(a, v) // b = min(b, v)
        if v >= b: return v // if v <= a:
    return v
```

Machine Learning and Decision Trees

Machine Learning

- Some problems are intractable: They do not have an efficient solution for all cases

- Other problems are difficult because formulating the rules in a way that computers can process is hard
- A learning agent learns a function to identify patterns in the data and make decisions rather than explicitly solving it
- Machine Learning is a subfield of AI where computers learn without being explicitly programmed
- Supervised: Learn from labeled data (input-output pairs) to find a mapping from inputs to outputs
- Unsupervised: Learn from unlabeled data (input only) to find patterns or structure
- There is also semi-supervised or reinforcement learning

Supervised Learning

- Data: Set of input-output (ground truth) pairs
- Model: Function that predicts output using input features
- Loss: Function used to assess model's performance
- Learning algorithm seeks to minimise loss of a model over a training dataset
- Model is evaluated on a test set using performance measures to estimate the generalisation of the model
- Classification: Predict a discrete label based on input features; output is a categorical value
- Regression: Predict a continuous numerical value based on input features; output is a real number
- A model or hypothesis refers to a function that maps from inputs to outputs and can be learned by a learning algorithm
- Performance measure is a metric evaluating how well the model performs
- A loss function quantifies the difference between the model's predictions and true outputs
- Examples of performance measures include mean squared error or mean absolute error for regression, and accuracy for classification
- For binary classification, we can look at the confusion matrix for true/false positives/negatives: "The prediction is [+/-] and it is [T/F]"
- $Accuracy = \frac{TP+TN}{TP+FN+FP+TN}$
- $Precision = \frac{TP}{TP+FP}$ if false positives are costly
- $Recall = \frac{TP}{TP+FN}$ if false negatives are costly
- $F1 = \frac{2}{1/P+1/R}$
- A learning algorithm takes a training set of input-output pairs and finds a model which approximates the true relationship between inputs and outputs

Decision Trees

- We are given a set of data points consisting of features and a target variable which are categorical
- A decision tree takes in an input vector of categorical attributes and outputs a predicted category
- Internal nodes represent tests on features, Branches correspond to outcomes of tests, Leaf nodes assign predictions
- A decision is made by checking the attribute value at each node and following it to the leaf node
- Prefer more compact decision trees, although they are less likely to be consistent with training data by chance
- Finding the optimal decision tree is NP-Hard, but it can be made tractable by greedy heuristics
- At each step, choose the feature that yields the greatest

- immediate information gain
- Entropy: Measure of uncertainty or randomness in a set of data, or number of bits used to represent it
- Entropy: $H(P(v_1), ...P(v_n)) = - \sum_{i=1}^n P(v_i)log_2P(v_i)$
- Common Entropies: $H(1) = 0, H(1/2, 1/2) = 1, H(1/3, 2/3) = 0.918, H(3/5, 2/5) = 0.971$
- For binary outcomes, $H(\frac{p}{p+n}, \frac{n}{p+n}) = -\frac{p}{p+n}log_2\frac{p}{p+n} - \frac{n}{p+n}log_2\frac{n}{p+n}$
- Specific Conditional Entropy: $H(Y|X = x)$ is the entropy of Y conditioned that $X = x$, defined as $H(Y|X = x) = H(P(y_1|x), ..., P(y_k|x))$, only looking at a subset of samples fulfilling a condition
- Conditional Entropy: $H(Y|X)$ is the entropy of Y when the value of X is known, defined as $H(Y|X) = \sum_{x \in X} P(X = x)H(Y|X = x)$
- Information Gain: Measure of reduction in uncertainty after knowing the value of a specific variable, defined as $IG(Y; X) = H(Y) - H(Y|X)$
- Choose attributes with the highest information gain
- Stop splitting when all rows have same classification or there are no more attributes
- Theorem (Representational Completeness): For any finite set of consistent examples with discrete features and a finite set of label classes, there exists a decision tree that is consistent with the examples
- However, noise or outliers might cause learned decision tree to be unnecessarily complex or overfitting
- Pruning can produce simpler trees e.g. Max depth, Min Sample Leaves
- Max Depth: Limits maximum depth / length of longest path from a root node to a leaf node
- Min Sample Leaves: Set a minimum threshold for the number of samples required to be in a leaf node
- Partition continuous valued attributes into discrete intervals
- For missing values, come up with a way to populate them e.g. most common value of attribute, or same thing but only those with same output etc.

Linear Regression

- Suppose each data point consists of features and a target variable, all of which are real numbers
- Given another data point without a target, we want to find a function that can predict a target for this new data point
- Linear Model: $h_w(x) = w_0x_0 + w_1x_1 + ... + w_dx_d$ where w_i are weights and $x_0 = 1$ is a dummy variable
- $h_w(x) = w^Tx = \sum_{j=0}^d w_jx_j$
- We want the linear model with the lowest loss on data, with a common measure being MSE
- MSE: $J_{MSE}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$, $\frac{\partial J_{MSE}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})x_j^{(i)}$
- The gradient of h is $\nabla h(w) = [\frac{\delta h(w)}{\delta w_1} ... \frac{\delta h(w)}{\delta w_d}]^T = [x_1 ... x_d]^T$
- To minimise a function $f(w)$, we find the point where $\nabla f(w) = 0$, so for our multidimensional function, we want $\frac{\delta f(w)}{\delta w_i} = 0$ for all w_i
- Minimum loss: $\nabla J_{MSE}(w) = 0$
- Normal Equation: Use $w = (X^TX)^{-1}X^TY$, derived

- from $X^TX(w - Y) = 0$
- However, the cost is $O(d^3)$ for inversion, and requires X^TX to be invertible (not possible if features > data points)
- Gradient Descent: Start at a random w , and update w with a step in opposite direction of gradient repeatedly until some termination criterion is satisfied
- $w_j = w_j - \gamma \frac{\delta J(w_0, ...)}{\delta w_j}$ where $\gamma > 0$ is the learning rate
- Convexity: For a line segment connecting any two distinct points on the graph, the line lies on or above the graph between two points
- Theorem: The MSE loss function is convex
- Linearly Dependent Features: A feature which can be expressed as a linear combination of other features
- Theorem: If the features in the training data are linearly independent, the function is strictly convex
- Theorem: For a linear regression model with MSE, gradient descent will converge to a global minimum provided the learning rate is chosen appropriately, which is unique if the features are linearly independent
- Learning Rate Scheduler: Decrease learning rate during training
- Larger features have greater updates to weights, so we need to scale them
- Min-max Scaling: $x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$
- Standardisation: $x'_i = \frac{x_i - \mu_i}{\sigma_i}$, using normal distribution
- Alternatively have different learning rate for each weight
- Gradient descent can be very slow since it requires the entire dataset; can get stuck at local minimum or plateau on non-convex optimisation
- Batch Gradient Descent: Consider all training examples
- Mini-Batch Gradient Descent: Consider a subset of training examples at a time, cheaper per iteration, noisy gradient can help escape local minima
- Stochastic Gradient Descent: Select a random training example at a time
- n points, needs maximum degree of $n - 1$

Feature Transformation

- Transform d dimensional feature into M dimensional, usually $M \geq d$ using a function e.g. polynomial, log, exp
- Theorem: The convexity of linear regression with MSE is preserved with feature transformation
- Model is usually $\hat{h}(x) = h(\phi(x))$, where ϕ is the feature transformation function and h is the predictor

Logistic Regression

- Given a set of data points with features and a target variable, but this time they are described by real numbers, and the target is either a positive or negative class (1 or 0)
- This is a classification problem (binary in this case)
- Sigmoid Function: $\sigma(x) = \frac{1}{1+e^{-x}}$, which is differentiable with derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- Logistic Model: Function of the form $h_w(x) = \sigma(w_0x_0 + ... + w_dx_d) = \sigma(w^Tx)$ where $x_0 = 1$ is a dummy variable
- The model's output is the probability of an input to be class 1, giving the confidence score of the model
- $\frac{\partial J_{BCE}(w)}{\partial w_j} = (h_w(x) - y)x_j$

- Example: Outputs $h_w(x) = P(Fail|x) = 0.8$ means it predicts the probability of failure to be **0.8**
- To decide which class an input belongs to, compare the probability output with a given threshold
- Decision Boundary: Surface that separates different classes in the feature space, which is intersection between model's function and decision threshold, $(w_i - w_j)^Tx = 0$
- Theorem: MSE is non-convex for logistic regression
- Cross-Entropy: Measures difference between two probability distributions over same set of outcomes, suppose P is the true probability distribution and Q is our model, then we have $H(P, Q) = - \sum_x P(x)log Q(x)$
- Binary Cross Entropy: Special case where there are only two possible outcomes, so $H(P, Q) = -P(1)log Q(1) - P(0)log Q(0)$
- BCE Loss: $J_{BCE}(w) = \frac{1}{n} \sum_{i=1}^n BCE(y^{(i)}, h_w(x^{(i)}))$
- Theorem: The BCE loss function is convex
- Partial derivative equation does not have a closed form
- Theorem: For a logistic model with BCE, gradient descent is guaranteed to converge to a global minimum provided the learning rate is chosen appropriately
- Data is linearly separable when a single linear decision boundary separates the different classes
- When it is not, the decision boundary required needs to be non-linear or more complex, which can be done with feature transformation
- Theorem: The convexity of a logistic model with BCE is preserved with feature transformation
- Multi-Class Classification: Either one vs one or one vs rest
- One vs One: Classifier for every pair of classes, need $K(K - 1)/2$ of them, class with most votes is selected
- One vs Rest: Treat all other classes as single combined class, need K of them, classifier with highest probability determines the class
- Multi-Label Classification: Labels might not be mutually exclusive
- Binary Relevance: Train one binary classifier per label
- Classifier Chain: Feed predictions of previous labels as features for subsequent labels which can capture dependencies although dependent on order
- Label Powerset: Treat each unique set of labels as a single multiclass label

Topics in Supervised learning

- Generalisation: Model's ability to perform on unseen data
- Dataset Quality: Data should be relevant and balanced, but might contain noise
- Having more data typically leads to better performance
- Model Complexity: Size and expressiveness of model functions; higher model complexity allows for more sophisticated modeling of relationships
- A simple model is good for a simple ground truth but bad for complex ground truth (high bias, underfitting), retraining often leads to same model (low variance)
- Complex model overfits with scarce data, while it can fit better when data is abundant (low bias), retraining can lead to vastly different models (high variance)
- High Complexity: Has high error on unseen data, which reduces and converges as data quantity exists

- Low Complexity: High error on both unseen and training data even as data quantity increases, but does not diverge as much
- As model complexity increases, error **decreases** on unseen then increases again followed by decreasing again due to overparameterisation
- Hyperparameters: Settings that control behaviour of training algorithm but are not learned e.g. learning rate, feature transformations
- Hyperparameter Tuning: Optimising hyperparameters of a machine learning model to improve its performance (using validation set for evaluation)
- Techniques: Grid search, Random search, Local search, Successive halving, Bayesian optimisation

Regularisation

- Overfitting: Model is too complex for underlying ground truth because it learns training data too well; including noise and fluctuations
- Often associated with large-magnitude weights
- L_p -norm: $||w||_p = (\sum_{j=0}^d |w_j|^p)^{\frac{1}{p}}$
- Regularisation: Introduce constraints to a loss function to prevent model from becoming too complex
- Augment loss function with regulariser, $J_{reg}(w) = J(w) + \lambda R(w), \lambda \geq 0$
- Common ones are $L_1(|w_j|), L_2(w_j^2), L_\infty(max|w_j|)$
- Lasso Regression: Linear regression + L_1 norm, Ridge Regression: Linear regression + L_2 norm, Elastic Net: Linear Regression + $L_1 + L_2$ norm
- The objective is still to minimise the regularised loss, which is when the derivative is 0
- Normal Equation: $w = (X^T X + \lambda I)^{-1} X^T Y$
- Theorem (Invertibility): For $n \times d$ X and $d \times d$ I , for any $\lambda > 0$, $X^T X + \lambda I$ is invertible
- Theorem (Solution Uniqueness): For any $\lambda > 0$, ridge regression has a unique closed form solution given by the normal equation above
- Theorem (L_2 induces weight shrinkage): For any $\lambda > 0$, the L_2 norm of the regularised weight vector is strictly smaller than the non-regularised weight vector assuming a non-zero solution
- Theorem (L_1 penalty): If a feature has non-zero weight, the L_1 regulariser will penalise the feature with a constant penalty of λ in terms of gradient
- Theorem (L_1 induces sparsity): For a feature to be included, its contribution to reducing the MSE loss must be exactly equal to the regularisation penalty λ in terms of gradient, it not its weight will be set to exactly zero rather than pay the penalty

Kernel Method

- Problem: A linear model will have an extremely large number of weights with respect to the number of features
- Primal Formulation has $\binom{k+d}{k}$ terms
- Theorem: For a linear model with weight vector w , the best weight vector minimising loss can be expressed as $w = \sum_{j=1}^n a_j x^{(j)}$, linear combination of training input
- Representer Theorem: A more generalised version where prediction is of form $f(w^T x)$
- Primal and Dual Formulation:

- $h_w(x) = w^T x = \sum_{j=1}^n a_j x^{(j)T} x = h_a(x)$
- For the dual formulation, we minimise the loss over a instead of w
- For primal formulation, number of weights is all terms of maximum degree, while for dual formulation, it is proportional to number of data points
- Dual formulation is $\sum_{j=1}^n a_j \langle x^{(j)}, x \rangle$ (inner product)
- Theorem: For any feature mapping ϕ , there must exist a function $k_\phi(u, v)$ that computes $\langle \phi(u), \phi(v) \rangle$ for any u, v ; this is the kernel
- With kernel, the dual formulation is $h_a^\phi(x) = \sum_{j=1}^n a_j k_\phi(x^{(j)}, x)$
- Kernel Trick: We do not have to expand $\phi(x)$, we can simply compute kernel between $x^{(j)}, x$
- Polynomial Degree k Kernel: $k_{pk}(u, v) = \phi_{pk}(u)^T \phi_{pk}(v) = (u^T v)^k$, feature map is polynomial combinations of features with degree $\leq k$
- Gaussian Kernel with variance s^2 : $k_{RBF}(u, v) = e^{-\frac{||u-v||^2}{2s^2}}$, feature map is an infinite list of scaled powers of the input features
- Mercer's Theorem: For any valid kernel k_ϕ , there must exist a vector space with a feature mapping ϕ

Support Vector Machines

- Classification problem, each data point has d features which are real numbers and the output is $-1, 1$ for a negative and positive class
- SVM Model: $h_w(x) = sign(w^T x)$
- Zero-offset hyperplane: Normal vector w containing all u where $w^T u = 0$
- Prediction: Use $sign(w^T x)$, tells us which side it lies on
- Distance to hyperplane: $\frac{|w^T x|}{||w||}$, LOP
- Margin: Distance between hyperplane and closest training data points, $\frac{1}{||w||}$, we aim to maximise this
- Support Vectors: Training data points on margin boundaries
- Dual Formulation: $h_a^\phi(x) = sign(\sum_{i=1}^n a_i k_\phi(x^{(i)}, x))$
- Theorem: For linearly separable data, the optimal dual-formulation SVM has $a_i > 0$ iff it lies exactly on the boundary (support vectors), and $a_i = 0$ if it is strictly outside the margin boundary
- Theorem: For linearly separable data with r dimensions, the number of support vectors is at least $r + 1$, in practice it is much less than the number of data points
- Due to sparsity (most coefficients are zeroed out), we only need to store support vectors and their corresponding a_i
- The SVM model is given by $h_a(x) = sign(\sum_{x^{(i)} \in S} a_i k(x^{(i)}, x))$
- Hyperplane is $w^T x + b = 0, b = -w^T x$, check sign

Unsupervised Learning

- Data where we do not have ground truths
- Clustering: Identify clusters in data
- Dimensionality Reduction: Find a lower-dimensional representation of the data while retaining relevant information

K-means Clustering

- Centroid: Average of a set of data points, $\mu_j = \frac{1}{n_j} \sum_{i=1}^{n_j} x^{(i)}$, defines a cluster
- Randomly initialise centroids, assign points to the nearest centroids, then repeatedly update centroid of each cluster until convergence
- Distortion: Average square distance of each sample to its assigned centroid, $\frac{1}{n} \sum_i ||x^{(i)} - \mu_c(i)||^2$
- Theorem: Each step never increases distortion
- Could get stuck in local minima, either rerun many times with random initialisation or choose initial centroids better e.g. maximising distance between them
- K-Mediods: Use actual data point as cluster center
- Elbow Method: Use k on elbow of the loss against k graph

Dimensionality Reduction

- Curse of Dimensionality: The number of samples needed to learn a class increases exponentially with number of features, $n = O(2^d)$
- Orthonormal Basis: Set of normalised vectors such that every vector can be expressed as a unique linear combination of the vectors in the set
- SVD: Decompose into $A = U \Sigma V^T$
- Theorem: WLOG, $d > n$, A is $d \times n$, then the factorisation above has $d \times d$ U with d orthonormal columns and rows, $d \times n$ Σ which is diagonal with non-increasing singular values, and $n \times n$ V with n orthonormal columns and rows
- U is the new orthonormal basis, Σ is the importance matrix, while V^T is the linear combination coefficients
- The n singular values tells us the importance of the new basis vectors
- A data point is a linear combination of basis vectors in u
- We keep the r most significant vectors in the basis
- $u^T v = \sum_{j=1}^d u_j v_j = z ||v||$ gives the LOP of u onto v times the norm of v
- $\tilde{U}$ defines the r dimensional subspace of the feature space, and for any feature vector x , $\tilde{U}^T x$ gives the linear combination coefficients in the new subspace
- So we have $r \times n$ $Z = \tilde{U}^T X^T$ which projects the data points onto the new lower-dimensional basis
- Theorem: For a fixed r , $\tilde{U} \tilde{U}^T \approx I$ with approximation error dependent on r , so we can reconstruct the data with $\tilde{X}^T = \tilde{U} Z, \tilde{X}^T = \tilde{U} \tilde{U}^T X^T \approx X^T$
- Mean Centered Data:
 - Compute mean feature vector over all samples, $\bar{x} = \frac{1}{n} \sum_{i=1}^n x^{(i)}$
 - Compute mean centered data points, $\hat{x}^{(i)} = x^{(i)} - \bar{x}$
 - Use the mean centered data matrix
- Variance: Expectation of $(X - E[X])^2$
- Theorem: Given a mean centered data matrix $\hat{X}^T$, the values $\frac{\sigma_i^2}{n-1}$ are variances of the data in the basis defined by the vectors $u^{(j)}$
- Theorem: Choose the minimum r so that $\frac{\sum_{i=1}^r \sigma_i^2}{\sum_{i=1}^n \sigma_i^2} \geq 0.99$, then using this r , the original and mean centered data points satisfy $\frac{\sum_{i=1}^n ||\hat{x}^{(i)} - \bar{x}^{(i)}||^2}{\sum_{i=1}^n ||\hat{x}^{(i)}||^2} \leq 0.01$
- for distortion

- PCA is a statistical application of SVD, capturing components that maximise the statistical variation of the data using sample covariance matrix
- PCA works well for natural images which have structured patterns and redundancy

Neural Networks

- Neuron: Computes weighted sum of input features and passes result through activation function, $z = \sum_{j=0}^d w_j x_j, \hat{y} = g(z)$
- Activation Functions
 - Sigmoid: $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$
 - Identity: $g(z) = z$
 - Tanh: $g(z) = tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
 - Relu: $g(z) = max(0, z)$
 - Leaky Relu: $g(z) = max(az, z)$
- Neurons can work as a learnable feature transformation function to produce a new feature as output
- Hidden Layer: Any layer between input and output layers
- Both hidden and output layers are fully-connected where every neuron is connected to every feature or neuron in the previous layer
- Multi-Layer Neural Networks with only fully-connected layers can also be called Multi-layer perceptrons
- Forward Propagation: Process by which input data passes through a neural network to generate the output
- Single-Layer Neural Network: $\hat{y} = g(W^T x)$, W 's rows are the inputs for each neuron, while the columns are number of layer's neurons / output variables
- For multi-layered, compose g and $W^T x$ repeatedly
- Universal Approximation Theorem: NN with at least one hidden layer and non-linear activation function can approximate any continuous function

Tasks

- Binary classification: Features are real numbers, while target is 0 or 1 for negative or positive class
- The hidden layers learn feature transformation, and the output layer has 1 neuron with the sigmoid activation function
- Multi-class classification: The output target is now a label in $[1, C]$
- One-hot encoding: Represent categorical labels as vectors of numbers, using a vector with 1 for the class and 0 everywhere else
- The output now has C neurons, using the softmax activation function, and we pick the class with the highest $\hat{y}_i$
- Softmax: Probability of belonging to class i is $g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \in [0, 1]$ where z_i is the weighted sum value computed in neuron i , output sums to 1
- Single-output regression: The target is a real number
- The output layer has 1 neuron with the activation function being identity/none/sigmoid/tanh/...
- Multi-output regression: The target is a vector with K real numbers
- Similar, but the output layer has K neurons now

Backpropagation

- Gradient Computation:

- Generate predicted value $\hat{y}$ for a given (x, y) with $\hat{y} = g(\sum_{j=0}^d w_j x_j)$
- Compute the loss e.g. MSE: $L = \frac{1}{2}(\hat{y} - y)^2$
- Let $z = \sum_{j=0}^d w_j x_j$, $\hat{y} = g(z)$, the gradient of loss function is $\frac{\partial L}{\partial a} = (\hat{y} - y)(g'(z))(w_j)$, multiply with gradient or weight as we go along, sum if needed
- Reversal:
 - Compute gradient of loss wrt to $\hat{y}$
 - Convert $g(z)$ to $g'(z)$ and treat $g'(z)$ as the weight associated with the corresponding edge
 - Reverse the graph and compute gradients

Convolutional Neural Networks

- Images are represented as a grid of pixels
- Grayscale has one channel, while RGB has 3
- Large number of parameters, reduce using convolution
- Kernel / Filter: Matrix of weights
- Overlay the kernel, compute weighted sum using it, then slide and repeat to get output feature map
- Stride: Step size between positions
- Padding: Putting extra pixels around border of input data
- $H = \lfloor \frac{\text{size} - \text{window size} + 2 \cdot \text{padding}}{\text{stride}} \rfloor + 1$
- Receptive Field: Region of input image that a particular neuron in the network is influenced by
- $r_i = r_{i-1} + (\text{kernel}_i - 1) \times j_{i-1}$, $j_i = j_{i-1} \times \text{stride}_i$, $r_0 = j_0 = 1$
- For multiple input channels use multidimensional kernel
- Multiple kernels give multidimensional feature map
- Convolutional Layer: Use convolution to extract features, then apply activation function
- Pooling Layer: Downsamples feature maps e.g. Max-Pool or Average Pool
- Cross-Entropy Loss: Between predicted output after softmax and one-hot encoded target, $CE(y^{(i)}, \hat{y}^{(i)}) = -\sum_{j=1}^C y_j^{(i)} \log(\hat{y}_j^{(i)})$

Recurrent Neural Networks

- Given data points consisting of a sentence and a target label for each word, predict whether each word is a verb
- One-hot encoding: Each vector has length similar to vocabulary size, 1 is position of respective word
- One-hot takes a lot of space, does not have semantic similarity, poor generalisation
- Input is one-hot encoding, while output is the predicted label for each word
- Need contextual information, cannot predict each label independently
- Hidden State: Memory storing information from the past, used to make prediction for the next input
- Time Step: Single iteration in RNN where it processes one element of a sequence
- $h^{[1]} = g^{[h]}((W^{[xh]})^T x^{[1]} + (W^{[hh]})^T h^{[0]})$, $\hat{y}^{[1]} = g^{[y]}((W^{[hy]})^T h^{[1]})$ performs an update using the current data point and the hidden state
- Many-To-Many RNN: Takes in a sequence of inputs to produce a sequence of outputs
- Each hidden state is used as input to the next RNN layer together with encoded input at that position
- Each RNN layer produces this hidden state to be used

- Hidden state is fed into FC layer to produce output
- Deep Recurrent Neural Networks: Multiple RNN Layers between input and FC layer, each layer has hidden state similar to above, higher layers can learn more general abstractions
- Sentiment Analysis: Classify a sentence into negative or positive (0 or 1)
- Many-To-One: Many RNN layers each taking the hidden state output from the previous, only last one connects to FC layer and output
- Image Captioning: Given feature vector of an image, generate a descriptive sentence in one-hot encoding
- One-To-Many: Similar to Many-To-Many, but each RNN after the first uses the previous FC layer output as input rather than the encoded input vector
- Question Answering: Provide answer to a question, input and output are one-hot encoded, length might be different
- Many-To-Many, but first few RNN layers do not use output as input, use encoded input instead, last few RNN layers have output and use previous output as input
- Issues: Overfitting, Gradient Vanishing or Exploding
- Early Stopping: Halt training when models performance on validation set begins to worsen
- Dropout: Make neural network less reliant on specific neurons, randomly set some neurons' output to 0
- Vanishing Gradient: Small gradients get zeroed out due to repeated multiplication, change activation function to solve
- Exploding Gradient: Large gradients get multiplied until they overflow, use gradient clipping to bound range

Attention

- For RNN, information from beginning of sequence must travel through long intermediate steps and might be degraded when it reaches the end
- Attention: Allows NN to selectively focus on relevant parts of its input data when producing an output
- Model like YouTube search
- Query: Represents what current position is trying to retrieve (Search Query)
- Key: Describe what each position offers in terms of information (Video Keyword)
- Value: Actual information retrieved when position is attended to (Video Content)
- Query is compared to all keys to determine relevance
- Relevance / attention score is used to retrieve weighted sum of all values; the relevant context
- Self-Attention Layer: Each input element can find relevant context from all elements in the same sequence
- Computation:
 1. Transform inputs into QKV using learnable matrices; $q^{[t]} = W^q x^{[t]}$, $k^{[t]} = W^k x^{[t]}$, $v^{[t]} = W^v x^{[t]}$, $d_q = d_k$, $x^{[t]} \in \mathbb{R}^d$
 2. Compute attention scores with Softmax; $a_{i1} = \frac{(k^{[i]})^T q^{[1]}}{\sqrt{d_k}}$, $k^{[t]} \in \mathbb{R}^{d_k}$, $a'_{i1} = \frac{e^{a_{i1}}}{\sum_{i=1}^T e^{a_{i1}}}$, T is the length of the sequence; a_{ij} is the amount that token i attends to token j
 3. Dot products are scaled to prevent softmax from being pushed into regions where it has extremely small gradients

4. Compute weighted sum of all values; $h^{[1]} = \sum_{i=1}^T a'_{i1} v^{[i]}$, this forms the hidden state
 5. $Q/K/V = W^{Q/K/V} X$, $A = \frac{Q^T K}{\sqrt{d_k}}$, $A' = \text{softmax}(A)$, $H = VA$
- Masked Self-Attention Layer: Each input element can find relevant context from previous elements and current element itself, similar but $a_{ij} = -\infty$ if $i > j$, can only use those coming before current time step
 - Cross-Attention Layer: Elements can find relevant context from another sequence, Key and Value use $z^{[t]}$ instead rather than $x^{[t]}$

Attention Neural Networks

- Replace RNN layer with hidden state chains with self-attention layer now, producing hidden state for each encoded input
- Does not consider position of a word
- Positional Encoding: Inject positional information into original input vectors, $x'^{[t]} = x^{[t]} + PE^{[t]}$
- $PE^{[t]}$ must be unique for each position, consistent (not affected by length of sequence), and bounded
- Large PE might drown out actual meaning
- Use $\sin$ or $\cos$ to generate kth element using $PE_k^{[t]} = \sin(\frac{t}{C^{\frac{k}{d}}})$ if $k\%2 = 0$, $\cos(\frac{t}{C^{\frac{k-1}{d}}})$ otherwise
- Attention NN: Many-to-Many: Inputs are positionally encoded, and BCE loss is taken on output
- ANN: Many-to-One: Uses CLS (Classification Token) vector prepended to sequence, used to capture summary information of the entire input sequence and minimise training loss, hidden state for it is used by FC layer to produce output
- ANN: One-to-Many: Begin vector added after encoded feature vector, should be added into vocabulary and encoded like other words, have masked self-attention layer between input and hidden state, so each input can find context from previous elements
- Teacher Forcing: Used for sequence prediction; input for next step is true target, not model's current prediction
- No waiting, all output can be generated simultaneously
- Not available during testing, can lead to train-test mismatch, can gradually reduce teacher forcing over time
- ANN: Many-to-Many: have encoder and self attention layer generating initial hidden states, then BEGIN token feeding into masked self attention layer
- Masked self attention layer feeds into hidden states which feed into cross attention layers into FC layer into output
- Outputs piped back into masked self-attention layer
- Encoder: Reads input sequence and compresses information into vectors
- Decoder: Takes context vectors from encoder and generates new output sequence element by element

Transformer

- Deep attention neural network based on encoder and decoder building blocks
- Feed Forward Network: FC Layer, ReLU, FC Layer
- Encoder Block: Self-attention layer, Feed forward network
- Decoder Block: Masked self-attention layer, Cross-attention layer, Feed-forward Network

- Encoder: Stack of encoder blocks, Decoder: Stack of decoder blocks
- Generative Pretrained Transformers (GPT) are build on transformer architecture

Unsupervised Learning with NN Dimension Reduction

- Autoencoder: NN which learns to compress data, consisting of encoder and decoder
- Encoder: Takes high dimensional input data and generates compressed data
- Decoder: Takes compressed data and attempts to reconstruct the original high-dimensional input data
- Must learn to keep input's important features in compressed data
- Compression / Reconstruction: Use layers that progressively get smaller / larger
- Use MSE loss, reconstruction loss measures how different reconstructed output is from original input

Model Pre-Training

- Given a set of images, classify them into subsets
- Train model exclusively on labeled subset, wastes unlabeled subset
- New task: Solvable using unlabeled input image, but challenging enough to force model to deeply understand input image, and yields useful learned understanding
- Rotation Prediction: Assign label based on which rotation is applied, trained on image and rotated image pairs
- If model is not aware of concepts in images, it would not be able to recognise rotation
- Train model on invented task, create neural network for image classification, train model with labeled subset
- Pre-training, for feature transformation, initialise with trained weights, prediction part is randomly initialised
- Contrastive Learning: Train model to contrast samples, pull feature vector of similar images together, push feature vector of dissimilar images apart so model can learn to distinguish objects
 1. Take input, create different version of it to form positive pairs
 2. Use other images to form negative pairs
 3. Use CNN to extract feature vector for each image
 4. Train model so that high similarity has large value and low similarity has small value
 5. $p^{(i)} = \text{sim}(z^{(i)}, z'^{(i)})/\tau, \exp(p^{(i)})$
 6. $n^{(ij)} = \text{sim}(z^{(i)}, z^{(j)})/\tau, \exp(n^{(ij)})$
 7. $\text{sim}(u, v) = \frac{u^T v}{\|u\| \|v\|}$, $-1 \leq \text{sim}(u, v) \leq 1$, τ is a constant scaling factor
 8. Minimise $\ell^{(i)} = -\log(\frac{\exp(p^{(i)})}{\exp(p^{(i)}) + \exp(n^{(ij)})})$
 9. The pre-trained model weights can be used in image classifier model
- Image Inpainting: Cut out random patch from an image and mask it, then ask model to fill in missing patch, model needs to understand the content of the image, learned weights on encoder can be used for image classification model initialisation
- Unlabeled Text: Given sequence of text, model needs to predict next word, by understanding grammar and syntax